

BÀI TOÁN THỎA MÃN RÀNG BUỘC (CSP)

CHƯƠNG 3, PHẦN 7 VÀ CHƯƠNG 4, PHẦN 4.4

Nội dung

- ◇ Các ví dụ về CSP
- ◇ Tìm kiếm tổng quát áp dụng cho CSP
- ◇ Quay lui (Backtracking)
- ◇ Kiểm tra tới (Forward checking)
- ◇ Heuristic cho CSP

Bài toán thỏa mãn ràng buộc (CSP)

Bài toán tìm kiếm chuẩn:

Trạng thái là một “hộp đen”—một cấu trúc dữ liệu cũ bất kỳ

hỗ trợ kiểm tra đích, đánh giá, sinh trạng thái kế tiếp

CSP (Constraint Satisfaction Problems):

Trạng thái được định nghĩa bởi các *biến* V_i có *giá trị* thuộc *miền giá trị* D_i

Kiểm tra đích là một tập hợp các *ràng buộc* quy định

các tổ hợp giá trị cho phép đối với các tập con của các biến

Ví dụ đơn giản về một *ngôn ngữ biểu diễn hình thức*

Cho phép các thuật toán *đa năng* hữu ích có sức mạnh lớn hơn các thuật toán tìm kiếm tiêu chuẩn

Ví dụ: Bài toán 4 quân hậu dưới dạng CSP

Giả sử mỗi cột có một quân hậu. Mỗi quân hậu sẽ nằm ở hàng nào?

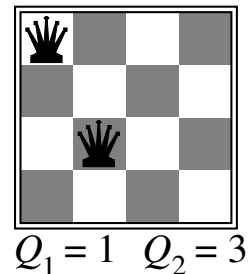
Các biến Q_1, Q_2, Q_3, Q_4

Miền giá trị $D_i = \{1, 2, 3, 4\}$

Các ràng buộc

$Q_i \neq Q_j$ (không thể ở cùng hàng)

$|Q_i - Q_j| \neq |i - j|$ (hoặc cùng đường chéo)



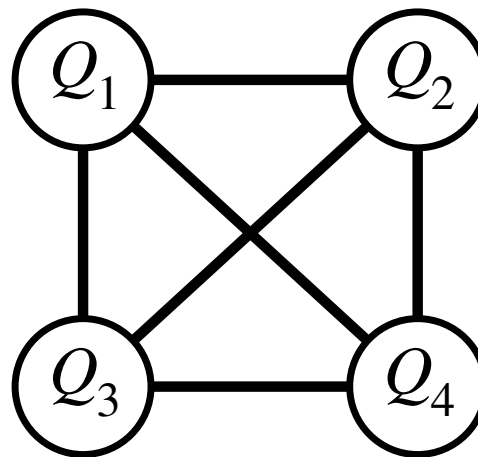
Dịch mỗi ràng buộc thành tập các giá trị cho phép đối với các biến của nó

Ví dụ: các giá trị cho (Q_1, Q_2) là $(1, 3)$ $(1, 4)$ $(2, 4)$ $(3, 1)$ $(4, 1)$ $(4, 2)$

Đồ thị ràng buộc

CSP nhị phân (Binary CSP): mỗi ràng buộc liên kết tối đa hai biến

Đồ thị ràng buộc: các nút là các biến, các cung thể hiện các ràng buộc



Ví dụ: Số học mật mã (Cryptarithmetic)

Các biến

$D E M N O R S Y$

Miền giá trị

$\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$

$$\begin{array}{r} S E N D \\ + M O R E \\ \hline M O N E Y \end{array}$$

Các ràng buộc

$M \neq 0, S \neq 0$ (ràng buộc một ngôi)

$Y = D + E$ hoặc $Y = D + E - 10$, v.v.

$D \neq E, D \neq M, D \neq N$, v.v.

Ví dụ: Tô màu bản đồ

Tô màu một bản đồ sao cho không có hai quốc gia kề nhau nào có cùng màu

Các biến

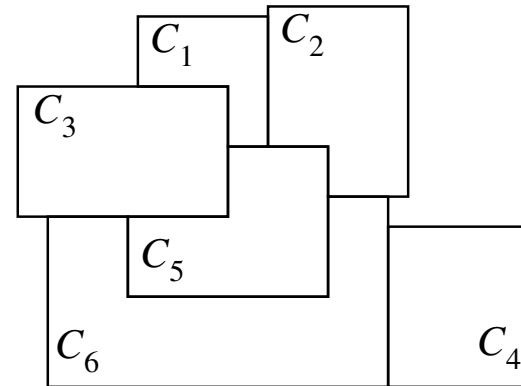
Các quốc gia C_i

Miền giá trị

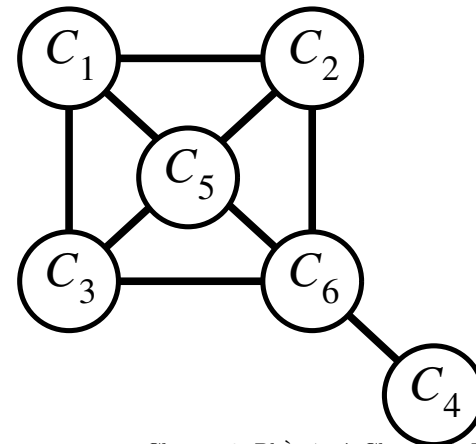
$\{\text{Đỏ}, \text{Xanh dương}, \text{Xanh lá}\}$

Các ràng buộc

$C_1 \neq C_2, C_1 \neq C_5$, v.v.



Đồ thị ràng buộc:



Các CSP trong thế giới thực

Bài toán phân công

ví dụ: ai dạy lớp nào

Bài toán xếp thời khóa biểu

ví dụ: lớp nào được cung cấp khi nào và ở đâu?

Cấu hình phần cứng

Bảng tính (Spreadsheets)

Lập lịch vận tải

Lập lịch nhà máy

Thiết kế sơ đồ mặt bằng (Floorplanning)

Lưu ý rằng nhiều bài toán trong thế giới thực liên quan đến các biến có giá trị thực

Áp dụng tìm kiếm tiêu chuẩn

Hãy bắt đầu với một phương pháp đơn giản, ngớ ngếch, sau đó sửa chữa nó

Các trạng thái được xác định bởi các giá trị đã được gán cho đến hiện tại

Trạng thái ban đầu: tất cả các biến đều chưa được gán

Toán tử: gán một giá trị cho một biến chưa được gán

Kiểm tra đích: tất cả các biến đã được gán, không có ràng buộc nào bị vi phạm

Lưu ý rằng điều này giống nhau đối với tất cả các CSP!

Cài đặt

Trạng thái CSP theo dõi các biến nào đã có giá trị cho đến thời điểm hiện tại

Mỗi biến có một miền và một giá trị hiện tại

datatype CSP-STATE

components: UNASSIGNED, danh sách các biến chưa được gán
ASSIGNED, danh sách các biến đã có giá trị

datatype CSP-VAR

components: NAME, phục vụ cho mục đích i/o
DOMAIN, danh sách các giá trị có thể
VALUE, giá trị hiện tại (nếu có)

Các ràng buộc có thể được biểu diễn

rõ ràng dưới dạng tập hợp các giá trị cho phép, hoặc

ngầm định bởi một hàm kiểm tra sự thỏa mãn ràng buộc

Tìm kiếm tiêu chuẩn áp dụng cho tô màu bản đồ



Độ phức tạp của phương pháp gốc ngược

◡ Độ sâu tối đa của không gian $m = ??$

◡ Độ sâu của trạng thái đích $d = ??$

◡ Thuật toán tìm kiếm sử dụng??

◡ Hệ số phân nhánh $b = ??$

Điều này có thể được cải thiện đáng kể bằng cách lưu ý những điều sau:

- 1) Thứ tự gán không quan trọng, do đó nhiều đường đi là tương đương nhau
- 2) Việc thêm các phép gán không thể khắc phục một ràng buộc đã bị vi phạm

Độ phức tạp của phương pháp gốc ngược

- ◡ Độ sâu tối đa của không gian $m = ??$ n (số lượng biến)
- ◡ Độ sâu của trạng thái đích $d = ??$ n (tất cả các biến đều được gán)
- ◡ Thuật toán tìm kiếm sử dụng?? tìm kiếm theo chiều sâu (depth-first)
- ◡ Hệ số phân nhánh $b = ??$ $\sum_i |D_i|$ (ở trên cùng của cây)

Điều này có thể được cải thiện đáng kể bằng cách lưu ý những điều sau:

- 1) Thứ tự gán không quan trọng, do đó nhiều đường đi là tương đương nhau
- 2) Việc thêm các phép gán không thể khắc phục một ràng buộc đã bị vi phạm

Tìm kiếm quay lui (Backtracking search)

Sử dụng tìm kiếm theo chiều sâu, nhưng

1) cố định thứ tự gán, $\Rightarrow b = |D_i|$

(có thể được thực hiện trong hàm SUCCESSORS)

2) kiểm tra sự vi phạm ràng buộc

Kiểm tra vi phạm ràng buộc có thể được thực hiện theo hai cách:

1) sửa đổi hàm SUCCESSORS để chỉ gán những giá trị

được phép, với những giá trị đã được gán trước đó

hoặc 2) kiểm tra các ràng buộc có được thỏa mãn trước khi phát triển

một trạng thái

Tìm kiếm quay lui là thuật toán không có thông tin cơ bản cho các CSP

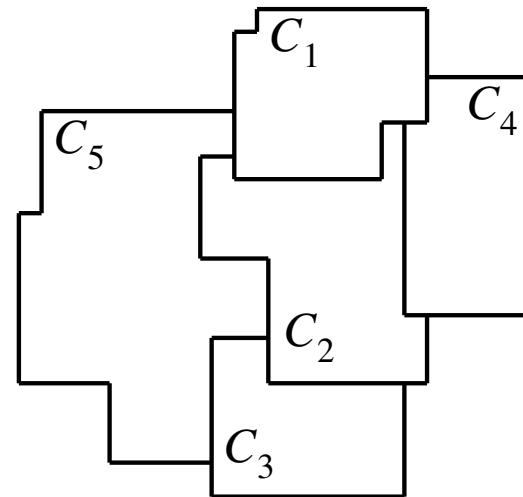
Có thể giải bài toán n -quân hậu với $n \approx 15$

Kiểm tra tới (Forward checking)

Ý tưởng: Theo dõi các giá trị hợp lệ còn lại cho các biến chưa được gán
Dừng tìm kiếm khi bất kỳ biến nào không còn giá trị hợp lệ nào

Ví dụ tô màu bản đồ được đơn giản hóa:

	ĐỎ	XANH DƯƠNG	XANH LÁ
C_1			
C_2			
C_3			
C_4			
C_5			



Có thể giải bài toán n -quân hậu lên đến $n \approx 30$

-
-
-

	✓		
	×		
	×		
	×		

-
-
-

		✓	
		×	
		×	
		×	

-
-
-

			✓
			✗

Heuristic cho CSP

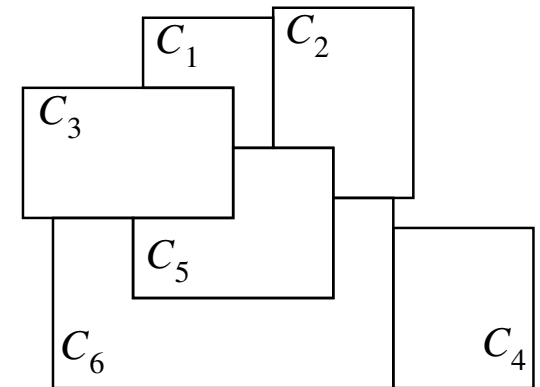
Các quyết định thông minh hơn về việc

chọn giá trị nào cho mỗi biến

biến nào sẽ được gán tiếp theo

• Cho $C_1 = \text{Đỏ}$, $C_2 = \text{Xanh lá}$, chọn $C_3 = ??$

• Cho $C_1 = \text{Đỏ}$, $C_2 = \text{Xanh lá}$, tiếp theo chọn biến nào??



Có thể giải bài toán n -quân hậu cho $n \approx 1000$

Heuristic cho CSP

Các quyết định thông minh hơn về việc

chọn giá trị nào cho mỗi biến

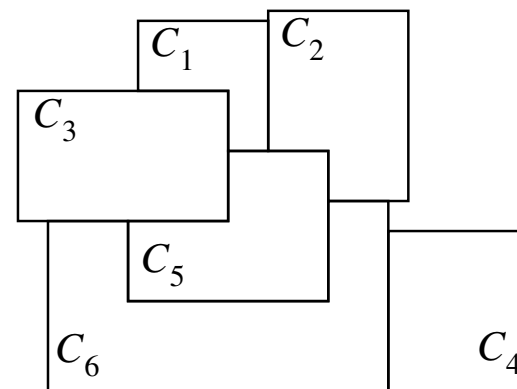
biến nào sẽ được gán tiếp theo

Cho $C_1 = \text{Đỏ}$, $C_2 = \text{Xanh lá}$, chọn $C_3 = ??$

$C_3 = \text{Xanh lá}$: *giá-trị-ràng-buộc-ít-nhất*
(*least-constraining-value*)

Cho $C_1 = \text{Đỏ}$, $C_2 = \text{Xanh lá}$, tiếp theo chọn biến nào??

C_5 : *biến-bị-ràng-buộc-nhiều-nhất* (*most-constrained-variable*)



Có thể giải bài toán n -quân hậu cho $n \approx 1000$

Thuật toán lặp cho CSP

Leo đồi, luyện kim mô phỏng thường hoạt động với các trạng thái “đầy đủ”, tức là tất cả các biến đã được gán

Để áp dụng cho CSP:

cho phép các trạng thái với các ràng buộc không được thỏa mãn

toán tử *gán lại* (reassign) các giá trị của biến

Lựa chọn biến: chọn ngẫu nhiên bất kỳ biến nào đang có xung đột

Heuristic *xung-đột-nhỏ-nhất* (min-conflicts):

chọn giá trị vi phạm ít ràng buộc nhất

tức là leo đồi với $h(n) =$ tổng số ràng buộc bị vi phạm

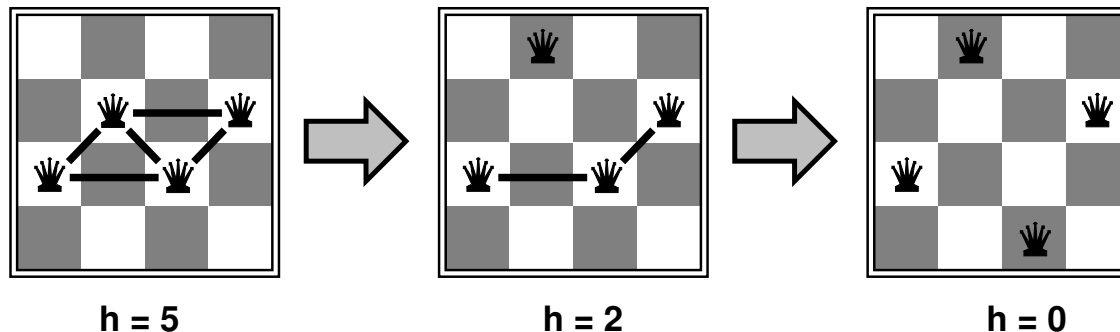
Ví dụ: Bài toán 4 quân hậu

Trạng thái: 4 quân hậu trong 4 cột (có $4^4 = 256$ trạng thái)

Toán tử: di chuyển quân hậu trong cột

Kiểm tra đích: không có sự tấn công nào

Đánh giá: $h(n) =$ số lượng các cuộc tấn công

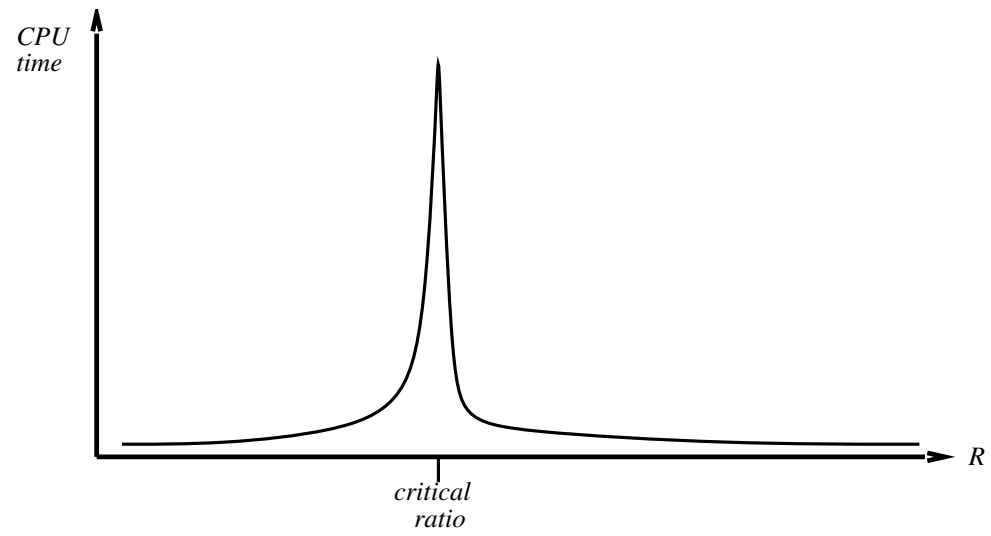


Hiệu suất của min-conflicts

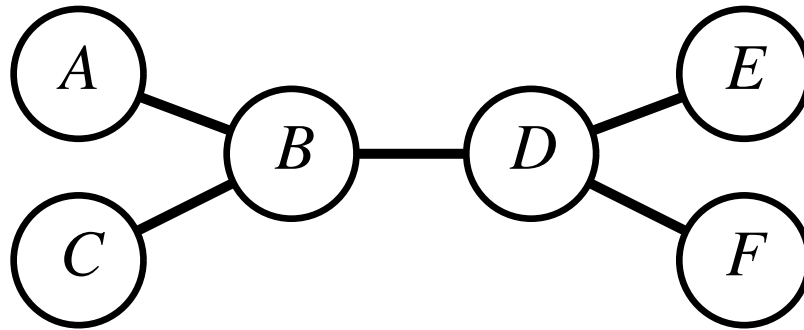
Với trạng thái khởi tạo ngẫu nhiên, có thể giải bài toán n -quân hậu trong thời gian gần như hằng số đối với bất kỳ n nào với xác suất cao (ví dụ: $n = 10,000,000$)

Điều tương tự có vẻ cũng đúng cho bất kỳ CSP được sinh ngẫu nhiên nào ngoại trừ trong một phạm vi hẹp của tỷ lệ

$$R = \frac{\text{số lượng ràng buộc}}{\text{số lượng biến}}$$



CSP có cấu trúc cây (Tree-structured CSPs)



Định lý: nếu đồ thị ràng buộc không có vòng lặp (loop), CSP có thể được giải trong thời gian $O(n|D|^2)$

So sánh với các CSP tổng quát, nơi thời gian xấu nhất là $O(|D|^n)$

Thuộc tính này cũng áp dụng cho suy luận logic và xác suất:
một ví dụ quan trọng về mối quan hệ giữa những hạn chế về cú pháp và độ phức tạp của suy luận.

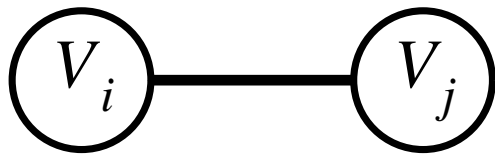
Thuật toán cho các CSP có cấu trúc cây

Bước cơ bản được gọi là *lọc* (*filtering*):

$Filter(V_i, V_j)$

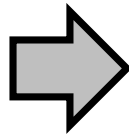
loại bỏ các giá trị của V_i không nhất quán với TẤT CẢ các giá trị của V_j

Ví dụ về lọc:



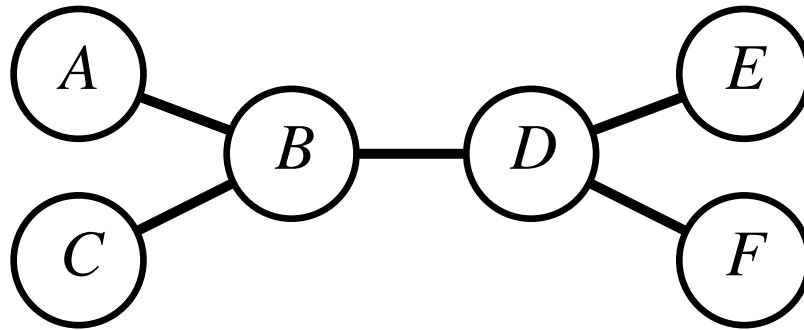
allowed pairs:

$\langle 1, 1 \rangle$
 $\langle 3, 2 \rangle$
 $\langle 3, 3 \rangle$

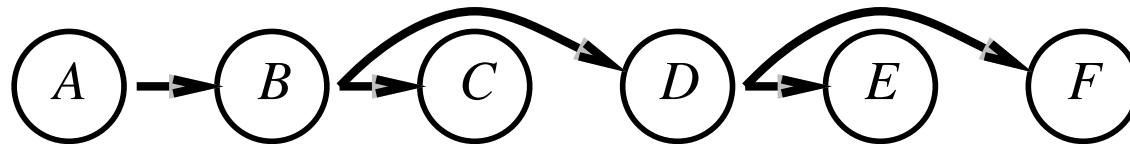


**remove 2 from
domain of V_i**

Thuật toán (tiếp)



1) Sắp xếp các nút theo chiều rộng (breadth-first) bắt đầu từ bất kỳ nút lá nào:



2) Đối với $j = n$ giảm xuống 1, áp dụng $Filter(V_i, V_j)$ trong đó V_i là nút cha của V_j

3) Đối với $j = 1$ tăng lên n , chọn giá trị hợp lệ cho V_j với giá trị của nút cha đã cho

Tóm tắt

CSP là một loại bài toán đặc biệt:

trạng thái được xác định bằng các giá trị của một tập hợp các biến cố định

kiểm tra đích được xác định bởi các *ràng buộc* trên giá trị của các biến

Quay lui (Backtracking) = tìm kiếm theo chiều sâu với

1) thứ tự biến cố định

2) chỉ có các trạng thái kế tiếp hợp lệ

Kiểm tra tới (Forward checking) ngăn chặn các phép gán chắc chắn dẫn đến thất bại sau này

Heuristic để chọn thứ tự biến và chọn giá trị giúp ích đáng kể

Cải tiến lặp min-conflicts thường có hiệu quả trong thực tế

Các CSP có cấu trúc cây luôn luôn có thể được giải quyết rất hiệu quả